

KRITIQ

The autonomous code review & fixer agent

Timeline	2-3 weeks
Stack	FastAPI · LangGraph · Groq · React
Storage	MongoDB Atlas
Deployment	Render.com · GitHub Pages
Cost	100% free

01 — OVERVIEW

What KRITIQ does

AGENT_01

Code reviewer

Fetches every file in a GitHub repo, analyzes each for bugs, code smells, and anti-patterns using Llama 3 via Groq. Flags issues with file path, line number, severity, and suggested fix.

AGENT_02

Security auditor

Runs in parallel with the reviewer. Scans for hardcoded secrets, SQL/command injections, insecure dependencies, and OWASP Top 10 patterns.

AGENT_03

Auto fixer

For each flagged issue, generates a validated fix, applies it to the file, and commits it back to the repo branch via PyGithub with a conventional commit message.

AGENT_04

README writer

Generates a production-grade README.md with project description, setup instructions, tech stack, live screenshots, and the auto-generated UML diagram embedded inline.

AGENT_05

Screenshot + UML

Captures a screenshot of the live deployed URL using Playwright headless Chromium. Generates Mermaid UML syntax and renders it to PNG, then commits both to the repo.

OUTPUT

Summary report

Produces a structured ReviewReport saved to MongoDB and streamed to the React dashboard. Posts a formatted review comment on the GitHub PR with all flags, fixes, and links.

Fully free · fully your stack

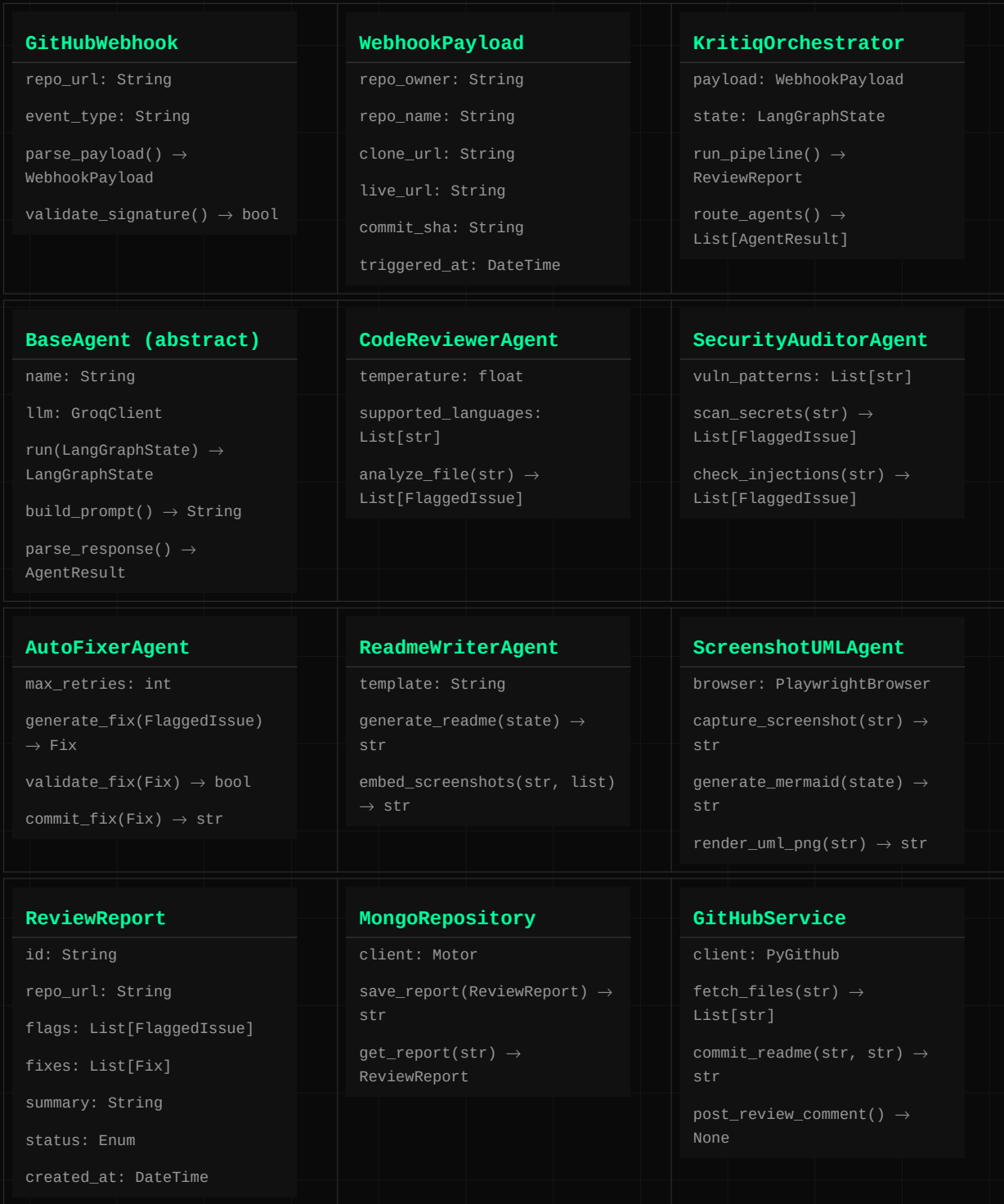
Agent core - Python 3.11 primary language - LangGraph state machine - LangChain LLM wrappers - Groq API Llama 3.3-70B free - Pydantic v2 all I/O validation	Backend - FastAPI webhook + API - Uvicorn ASGI server - Motor async MongoDB - PyGithub GitHub REST API - python-dotenv env management	Capture - Playwright screenshots + UML - Mermaid CLI UML PNG render - Chromium headless browser
Frontend - React 18 + Vite dashboard - TypeScript strict mode - Tailwind CSS styling - Zustand global state - TanStack Query data fetching	Storage - MongoDB Atlas free 512MB - Motor async Python driver	Deploy - Docker containerize API - Render.com free backend host - GitHub Pages free frontend host - GitHub Actions CI trigger

Three views of the system

UML 01 – CLASS DIAGRAM

Domain model & relationships

12 classes. BaseAgent is abstract — 5 agents extend it. ReviewReport is the central output aggregating all flags, fixes, screenshots, and UML path. MongoRepository and GitHubService are infrastructure adapters kept outside the agent layer.



UML 02 – SEQUENCE DIAGRAM

Message flow across all participants

24 numbered steps from GitHub webhook to final 200 OK. The alt/else block models the conditional fixer path — if no issues are flagged, AutoFixerAgent is skipped.

1. GitHub → FastAPI: POST /webhook (push or PR event)
2. FastAPI → FastAPI: validate_signature(secret)

```

3. FastAPI → Orchestrator: trigger_pipeline(WebhookPayload)

4. Orchestrator → GitHubService: fetch_files(repo_url, commit_sha)

5. GitHubService → Orchestrator: List[FileContent]

6. Orchestrator → CodeReviewerAgent: run(LangGraphState)

7. CodeReviewerAgent → CodeReviewerAgent: analyze_file() per file

8. CodeReviewerAgent → Orchestrator: LangGraphState + List[FlaggedIssue]

9. Orchestrator → SecurityAuditorAgent: run(LangGraphState)

10. SecurityAuditorAgent → SecurityAuditorAgent: scan_secrets() +
    check_injections()

11. SecurityAuditorAgent → Orchestrator: LangGraphState + security FlaggedIssues

[ALT: issues found]

12. Orchestrator → AutoFixerAgent: run(LangGraphState)

13. AutoFixerAgent → AutoFixerAgent: generate_fix(FlaggedIssue) per issue

14. AutoFixerAgent → GitHubService: commit_fix(Fix)

15. GitHubService → AutoFixerAgent: commit_sha

16. AutoFixerAgent → Orchestrator: LangGraphState + List[Fix]

[ELSE: no issues → skip AutoFixerAgent]

17. Orchestrator → ScreenshotUMLAgent: run(LangGraphState)

18. ScreenshotUMLAgent → ScreenshotUMLAgent: generate_mermaid + render_uml_png +
    capture_screenshot

19. ScreenshotUMLAgent → Orchestrator: + screenshot_paths + uml_path

20. Orchestrator → ReadmeWriterAgent: run(LangGraphState)

21. ReadmeWriterAgent → ReadmeWriterAgent: generate_readme + embed_screenshots +
    embed_uml

22. Orchestrator → GitHubService: commit_readme + commit_uml +
    post_review_comment

23. Orchestrator → MongoDB: save_report(ReviewReport)

24. FastAPI → GitHub: 200 OK + report_id

```

UML 03 – ACTIVITY DIAGRAM

Decision logic & pipeline flow

Full branching logic. CodeReviewer and SecurityAuditor fork in parallel. AutoFixer has a retry loop for invalid fixes. Screenshot capture is conditional on live URL being provided.

```

START: GitHub push or PR event

→ FastAPI receives webhook

```


Week-by-week execution plan

WEEK

01

Days 1–7

Foundation — repo setup, GitHub integration, base agents

- Day 1–2: Init monorepo — backend/ FastAPI skeleton, frontend/ Vite + React, docker-compose.yml, .env.example
- Day 2–3: GitHub webhook endpoint — validate HMAC signature, parse WebhookPayload with Pydantic, test with ngrok locally
- Day 3–4: GitHubService — fetch all repo files via PyGithub, filter .gitignore patterns, commit back helper
- Day 4–5: BaseAgent abstract class + CodeReviewerAgent — Groq client, streaming prompt, Pydantic output parsing
- Day 5–6: SecurityAuditorAgent — secret patterns, injection checks, parallel run with reviewer via LangGraph fork
- Day 6–7: MongoDB Atlas setup, MongoRepository with Motor async, save and retrieve FlaggedIssue documents

✓ Deliverable — webhook triggers, files fetched, flags saved to MongoDB

WEEK

02

Days 8–14

Core agents — fixer, screenshot, UML, README writer

- Day 8–9: AutoFixerAgent — generate fix prompt, validate with retry loop (max 3), commit fix via GitHubService
- Day 9–10: LangGraph full state machine — wire all 5 agents, implement conditional routing, test end-to-end pipeline locally
- Day 10–11: ScreenshotUMLAgent — Playwright install, capture_screenshot() with full-page PNG, generate_mermaid() via Llama 3
- Day 11–12: Mermaid CLI render pipeline — LLM outputs Mermaid syntax → mmdc CLI renders PNG → commit to docs/uml.png
- Day 12–13: ReadmeWriterAgent — full README template, embed screenshots and UML, commit as README.md
- Day 13–14: Post GitHub PR review comment — structured markdown table of flags + fixes + links via PyGithub

✓ Deliverable — full pipeline runs end-to-end on a real GitHub repo

WEEK

03

Days 15–21

Dashboard, deployment, polish

Day 15–16: React dashboard — repo input, trigger KRITIQ manually, Zustand store for agent step state, TanStack Query polling

Day 16–17: Dashboard UI — live agent progress stepper, flagged issues table, fixes diff view, dark luxury aesthetic with Tailwind

Day 17–18: FastAPI streaming endpoint — GET /pipeline/stream using StreamingResponse + SSE, React useEventSource hook

Day 18–19: Dockerfile — multi-stage build, install Playwright + Chromium inside container, test on Render.com free tier

Day 19–20: GitHub Actions CI — on push to main: run tests, build Docker image, deploy to Render.com; deploy React to GitHub Pages

Day 20–21: Final testing on 3 real repos (Python, Node, mixed), README polish, record demo GIF, add to portfolio

✓ Deliverable – KRITIQ live at Render URL, dashboard on GitHub Pages, tested on real repos

05 – PROJECT STRUCTURE

Folder layout

```
kritiq/  
  ■■■ backend/  
    ■ ■■■ app/  
      ■ ■ ■■■ api/  
        ■ ■ ■ ■■■ v1/webhooks.py # GitHub webhook handler  
        ■ ■ ■ ■■■ v1/reports.py # ReviewReport CRUD + stream  
      ■ ■ ■■■ agents/  
        ■ ■ ■ ■■■ base.py # BaseAgent abstract class  
        ■ ■ ■ ■■■ code_reviewer.py  
        ■ ■ ■ ■■■ security_auditor.py  
        ■ ■ ■ ■■■ auto_fixer.py  
        ■ ■ ■ ■■■ readme_writer.py  
        ■ ■ ■ ■■■ screenshot_uml.py  
      ■ ■ ■■■ core/
```

```

■ ■ ■ ■■ config.py # pydantic-settings
■ ■ ■ ■■ orchestrator.py # LangGraph state machine
■ ■ ■■■ services/
■ ■ ■ ■■ github_service.py # PyGithub wrapper
■ ■ ■ ■■ mongo_repository.py # Motor async MongoDB
■ ■ ■■■ schemas/
■ ■ ■ ■■ webhook.py # WebhookPayload
■ ■ ■ ■■ issue.py # FlaggedIssue, Fix
■ ■ ■ ■■ report.py # ReviewReport
■ ■ ■■ main.py # FastAPI app, lifespan, routers
■ ■■■ Dockerfile
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/ # AgentStepper, IssueTable, DiffView
■ ■ ■■■ pages/ # Dashboard.tsx, Report.tsx
■ ■ ■■■ hooks/ # useEventSource, useReport
■ ■ ■■■ store/ # Zustand pipeline state
■ ■ ■■■ types/ # ReviewReport, FlaggedIssue TS types
■ ■■■ vite.config.ts
■ ■■■ package.json
■■■ .github/
■ ■■■ workflows/
■ ■■■ deploy.yml # CI – test, build, deploy
■■■ docker-compose.yml
■■■ .env.example
■■■ README.md # auto-generated by KRITIQ itself

```

06 – API ENDPOINTS

FastAPI routes

Endpoint	Method	Description
/webhook/github	POST	Receives GitHub push/PR webhook, validates HMAC, triggers pipeline

<code>/pipeline/run</code>	POST	Manual trigger — accepts <code>repo_url</code> + optional <code>live_url</code> , runs full pipeline
<code>/pipeline/stream/{report_id}</code>	GET	SSE stream — emits agent step events as pipeline runs in background
<code>/reports</code>	GET	List all ReviewReports for a repo, paginated, sorted by <code>created_at</code> desc
<code>/reports/{report_id}</code>	GET	Fetch a single ReviewReport with all flags, fixes, paths
<code>/reports/{report_id}/summary</code>	GET	Returns LLM-generated plain-English summary of the review
<code>/health</code>	GET	Health check — returns status of MongoDB, Groq API, GitHub token

07 — MONGODB SCHEMA

Collections layout

Field	Type	Description
<code>_id</code>	ObjectId	auto-generated
<code>repo_url</code>	String	GitHub repo full URL
<code>commit_sha</code>	String	SHA that triggered the review
<code>flags</code>	Array[FlaggedIssue]	embedded documents
<code>fixes</code>	Array[Fix]	embedded documents
<code>readme_path</code>	String	path in repo to committed README
<code>screenshot_paths</code>	Array[String]	paths in repo to committed PNGs
<code>uml_path</code>	String	path in repo to committed UML PNG
<code>summary</code>	String	LLM-generated plain-English summary
<code>status</code>	Enum	pending · running · complete · failed
<code>created_at</code>	DateTime	UTC timestamp

Field	Type	Description
id	String	uuid4
file_path	String	relative path in repo
line_number	Int	line where issue starts
severity	Enum	critical · high · medium · low · info
category	Enum	security · bug · style · performance · docs
description	String	human-readable explanation
suggested_fix	String	LLM suggested fix before auto-fixer runs

08 – ENVIRONMENT VARIABLES

.env.example

```
# Groq (free tier – llama-3.3-70b-versatile)
GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXX

# GitHub Personal Access Token (repo + webhook scope)
GITHUB_TOKEN=ghp_XXXXXXXXXXXXXXXXXXXX

GITHUB_WEBHOOK_SECRET=your_webhook_secret

# MongoDB Atlas (free cluster)
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/kritiq
MONGODB_DB_NAME=kritiq

# App
APP_ENV=development

CORS_ORIGINS=http://localhost:5173
```